

Adapt — Design Document

MPPI Motion Planner for the UIUC Polaris GEM e4

Sunny Deshpande — UIUC AVSE

2026-04-17

Contents

1	Executive Summary	3
2	Project Background	4
2.1	What Adapt is	4
2.2	Stack position	4
2.3	Why MPPI (and not Stanley / QP-MPC / iLQR)	4
2.4	Where the GEM Gazebo sim fits	5
3	System Architecture	6
3.1	ROS 2 topic contract (Phase 1 node)	6
3.2	Vehicle model	6
3.3	Reference path	6
3.4	MPPI algorithm	7
3.5	Cost function	7
3.6	Default parameters	7
3.7	Software layout	8
4	Current Progress	9
4.1	Phase 1 — MPPI controller (complete, verified)	9
4.2	Sim-bridge infrastructure (scaffolded 2026-04-17)	9
4.3	Key paths (host vs. container)	9
5	How to Run	11
5.1	Standalone MPPI test (no ROS, no sim)	11
5.2	Phase 1 as a ROS 2 node (Humble, no Gazebo)	11
5.3	GEM Gazebo sim + MPPI bridge (Docker Noetic)	11
6	What to Do Next	13
6.1	Near term (finish the closed-loop path)	13
6.2	Medium term	13
6.3	Next design phase (Phase 2 proper)	13
6.4	Long term	13
6.5	Aspirational research track	14
6.6	Known risks to revisit	14

6.7 Tuning guide (inherited from Phase 1) 14

1 Executive Summary

Adapt replaces AutoShield's Stanley lateral controller on the UIUC Polaris GEM e4 with a **Model Predictive Path Integral (MPPI)** motion planner. MPPI samples hundreds of candidate control sequences per tick, weighs them by a cost function, and emits a softmax-averaged update. It reasons jointly about steering **and** acceleration, and natively accepts non-differentiable costs — notably, obstacle clearance costs derived from AutoShield's pedestrian prediction pipeline.

Current status (2026-04-17):

- **Phase 1 — MPPI controller: complete.** Standalone numerical test: mean $|\text{lateral error}| = 0.007$ m, max 0.024 m, 100 % of post-warmup steps within the 0.5 m spec. ROS2 node at 10 Hz, drop-in compatible with AutoShield's PACMod topics.
- **Sim-bridge infrastructure: scaffolded, not yet run closed-loop.** A thin ROS1 bridge exposes the pure-numpy MPPI to the POLARIS GEM Gazebo simulator (Noetic, Docker).
- **Next:** first closed-loop run in `track1.world` on the E4, then pedestrian-prediction integration (shape change of the obstacle cost from static (M, 2) to time-indexed (M, H, 2)).

2 Project Background

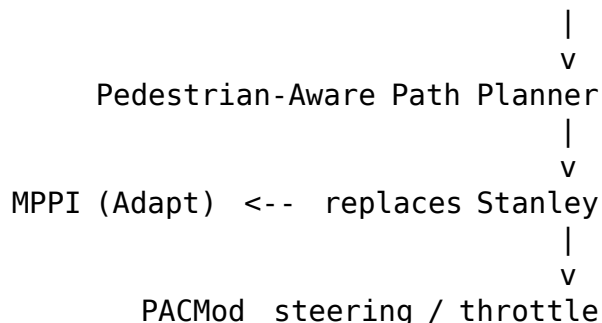
2.1 What Adapt is

A UIUC AVSE research project retrofitting AutoShield’s autonomy stack to evaluate sampling-based MPC (MPPI) as the production motion planner on the Polaris GEM e4. Deliverables are both a working controller and a reproducible evaluation pipeline (standalone test → Gazebo sim → real vehicle).

A parallel research track explores replacing AutoShield’s current pedestrian-motion predictor with a **diffusion policy** that generates multi-modal future trajectories. Whether this lands in the current project window is uncertain; the MPPI controller is designed so the prediction source can be swapped without touching the planner — the obstacle-cost tensor just changes shape from $(M, 2)$ to $(M, H, 2)$ (see §Next design phase).

2.2 Stack position

LiDAR / RGB-D --> Sensor Fusion --> Pedestrian Prediction --> High-Level Decision



Adapt touches only the controller node. Upstream (perception, prediction, path planner) and downstream (PACMod command topics) are preserved.

2.3 Why MPPI (and not Stanley / QP-MPC / iLQR)

Stanley is reactive (no prediction), steering-only (longitudinal is a decoupled PID), and every new safety consideration requires re-deriving gains. Linear MPC needs convex costs, which rules out the clipped quadratic clearance penalty used for obstacles. iLQR is on the edge of real-time at $H=30$ on CPU.

MPPI wins because:

1. **Non-smooth costs are trivial.** Obstacle cost is zero outside the clearance radius and quadratic inside — perfectly fine for sampling.
2. **CPU-friendly at scale.** The same NumPy broadcast that rolls out $K=600$ candidates also evaluates their costs. Wall time per tick ≈ 15 ms on a laptop CPU.
3. **Composable costs.** Future terms (time-indexed pedestrian predictions, uncertainty-weighted clearances) are absorbed without re-tuning anything — only weight *ratios* matter.

2.4 Where the GEM Gazebo sim fits

Phase 1 was verified with a standalone numerical test (no physics engine, no ROS). Before real-vehicle deployment, we need closed-loop evaluation under a full rigid-body simulator with realistic actuator lag and sensor timing. The **POLARIS GEM Simulator** (UIUC, ROS1 Noetic + Gazebo, shipped as a Docker image) fills that role. A thin bridge package exposes the pure-numpy MPPI to the ROS1 sim.

3 System Architecture

3.1 ROS 2 topic contract (Phase 1 node)

Direction	Topic	Type	Notes
in	/odom	nav_msgs/Odometry	Pose and body-frame velocity
in	/adapt/reference_path	nav_msgs/Path	Waypoints from AutoShield's path planner
in	/obstacles	geometry_msgs/PolygonStamped	Obstacles — each Point32 is one obstacle
out	/pacmod/steering_cmd	std_msgs/Float64	Steering angle (rad)
out	/pacmod/accel_cmd	std_msgs/Float64	Acceleration (m/s ²)

3.2 Vehicle model

Kinematic bicycle with state $\mathbf{x} = (x, y, \psi, v)$ and input $\mathbf{u} = (\delta, a)$. Euler integration at $\Delta t = 0.1$ s, wheelbase $L = 1.75$ m:

$$\begin{aligned}
 x_{k+1} &= x_k + v_k \cos \psi_k \Delta t \\
 y_{k+1} &= y_k + v_k \sin \psi_k \Delta t \\
 \psi_{k+1} &= \text{wrap}_{\pi}(\psi_k + (v_k/L) \tan \delta_k \Delta t) \\
 v_{k+1} &= \text{clip}(v_k + a_k \Delta t, 0, 8).
 \end{aligned}$$

`bicycle_model.step(states, controls)` accepts $(K, 4)$ and $(K, 2)$ arrays and returns $(\tilde{K}, 4)$ in one vectorized call — no Python-level loop over samples. Kinematic (not dynamic) is justified at 0–8 m/s where tire slip is $< 2^\circ$; AutoShield's Stanley also assumes kinematic.

3.3 Reference path

ReferencePath consumes waypoints $(x_i, y_i)_{i=0}^{N-1}$ and computes:

- cumulative arc length s_i ,
- per-segment heading $\theta_i = \text{atan2}(\Delta y_i, \Delta x_i)$,
- a vectorized `nearest_point(p)` that projects a query point onto **every** segment at once and picks the closest.

The signed lateral error is $d = (p_x - \hat{p}_x)(-\sin \theta) + (p_y - \hat{p}_y) \cos \theta$, used directly as the MPPI lane-deviation cost term.

3.4 MPPI algorithm

Per-tick update (maintained state: nominal control sequence $U \in \mathbb{R}^{H \times 2}$):

1. **Sample noise** $\varepsilon_{k,h} \sim \mathcal{N}(0, \Sigma)$, form $V = U + \varepsilon$, clip to actuator limits.
2. **Rollout** V through the batched bicycle step for H sequential calls, producing trajectory tensor of shape $(K, H, 4)$.
3. **Cost** $\mathcal{J}_k$ (five terms, see below), fully broadcast.
4. **Weights** (stable softmax): $\beta = \min_k \mathcal{J}_k$, $w_k = \exp(-(\mathcal{J}_k - \beta)/\lambda)/Z$.
5. **Update** $U \leftarrow U + \sum_k w_k \varepsilon_k$ (implemented as `np.einsum('k,kht->ht', w, noise)`).
6. **Emit** $u_0 = U_0$; receding-horizon shift $U \leftarrow \text{roll}(U, -1)$.

The warm-start U is what makes sampling MPC feasible at 10 Hz with only 600 samples; a cold-start MPPI needs $\sim 10\times$ more.

3.5 Cost function

$$\mathcal{J}_k = \sum_{h=0}^{H-1} \left[w_{\text{lat}} d_{k,h}^2 + w_{\text{head}} \tilde{\psi}_{k,h}^2 + w_{\text{spd}} (v_{k,h} - v_{\text{ref}})^2 + w_{\text{obs}} C_{k,h}^{\text{obs}} + w_{\text{ctrl}} \|u_{k,h}\|^2 \right]$$

with $C_{k,h}^{\text{obs}} = \sum_m (\max(r_c - \|p_{k,h} - o_m\|, 0))^2$ — quadratic-inside-clearance, zero outside. Computed as a single 4-D broadcast `diff = traj[:, :, None, :2] - obs[None, None, :, :]` (shape $(K, H, M, 2)$) — vectorized across all rollouts **and** all obstacles simultaneously.

3.6 Default parameters

Symbol	Default	Meaning
K	600	sample count
H	30	horizon (3.0 s lookahead)
Δt	0.1 s	rollout step
σ_δ	0.05 rad	steering noise std
σ_a	0.80 m/s ²	accel noise std
λ	1.0	softmax temperature
v_{ref}	3.0 m/s	cruise target
$w_{\text{lat}}, w_{\text{head}}, w_{\text{spd}}, w_{\text{obs}}, w_{\text{ctrl}}$	10, 5, 2, 1000, 0.1	cost weights
r_c	3.0 m	obstacle clearance radius
δ_{max}	0.61 rad	steering limit
$a_{\text{min}}, a_{\text{max}}$	-3, +2 m/s ²	accel limits

Only weight *ratios* matter — scaling every w by a constant is absorbed by λ .

3.7 Software layout

```
~/UIUC/AVSE/
├── Adapt/
│   ├── docs/                ← this document
│   ├── src/adapt_mppi/      ← ROS 2 package
│   │   └── adapt_mppi/
│   │       ├── bicycle_model.py    (batched step, wrap_to_pi)
│   │       ├── reference_path.py   (arc-length path, nearest_point)
│   │       ├── mppi.py             (MPPI class: update, costs, weights)
│   │       └── adapt_mppi_node.py   (ROS 2 node at 10 Hz)
│   ├── POLARIS_GEM_Simulator/     ← upstream sim repo (ROS 1 Noetic, Docker)
│   │   ├── vehicle_drivers/gem_mppi_sim/ ← sim-bridge package (ROS 1)
│   │   ├── scripts/mppi_sim.py
│   │   ├── launch/mppi_sim.launch
│   │   └── waypoints/wps.csv
│   └── CS588-SP26/                ← coursework, unrelated
```

`mppi.py`, `reference_path.py`, `bicycle_model.py` are pure NumPy — no ROS imports — so the sim bridge (ROS 1) reuses them unchanged by inserting `Adapt/src/adapt_mppi` into `sys.path` at startup.

4 Current Progress

4.1 Phase 1 — MPPI controller (complete, verified)

Standalone test (test/test_phase1.py) — 50 m radius arc, two static obstacles, 200 steps at $\Delta t = 0.1$ s. Runs without ROS.

Metric	Value
mean $ d $ (post-warmup)	0.007 m
max $ d $ (post-warmup)	0.024 m
fraction within 0.5 m	100 %
final speed	3.01 m/s (tracks $v_{\text{ref}} = 3$)

Pass criterion (80 % within 0.5 m after the first 20 warmup steps) is comfortably met. Vectorization invariants hold: the one for `h in range(H)` loop calls the batched bicycle step on a $(K, 4)$ batch once; cost evaluation and weight update have zero Python loops over K .

4.2 Sim-bridge infrastructure (scaffolded 2026-04-17)

ROS 1 catkin package `gem_mppi_sim` at `POLARIS_GEM_Simulator/vehicle_drivers/gem_mppi_sim`

- **State in:** `/gazebo/get_model_state` (model name `gem_e4`); fallback `/odom` via `use_odom:=true`.
- **Reference path:** CSV $\rightarrow$ `ReferencePath`. Default waypoints/`wps.csv` (track1 coordinates, copied from the pure-pursuit demo; vehicle-agnostic).
- **Control out:** `/ackermann_cmd` (`ackermann_msgs/AckermannDrive`). MPPI returns (δ, a) ; bridge integrates to a speed setpoint $v_{\text{cmd}} = \text{clip}(v + a \cdot dt, 0, 8)$ because the sim's existing `gem_control.py` expects a speed, not an acceleration.
- **Rate:** default 20 Hz.
- **Layout decision:** repo is **symlinked** into `~/gem_simulation_ws/src/` rather than moved, preserving the `~/UIUC/AVSE/{Adapt, POLARIS_GEM_Simulator, CS588-SP26}` sibling tree.

Pending: first closed-loop run on `track1.world`. Host-side one-time prep (workspace symlink + Docker image build) has not yet been executed.

4.3 Key paths (host vs. container)

What	Host	Container
Sim repo	<code>~/UIUC/AVSE/POLARIS_GEM_Simulator</code>	<code>~/host/UIUC/AVSE/POLARIS_GEM_Simulator</code>
Catkin workspace (symlinked)	<code>~/gem_simulation_ws/</code>	<code>~/host/gem_simulation_ws/</code>
Adapt MPPI	<code>~/UIUC/AVSE/Adapt/src/adapt_mppi</code>	<code>~/host/UIUC/AVSE/Adapt/src/adapt_mppi</code>
Bridge package	<code>POLARIS_GEM_Simulator/vehicle_drivers/gem_mppi_sim/</code>	<code>~/host/vehicle_drivers/gem_mppi_sim/</code>

Host-container path difference comes from the `${HOME}:/home/${USER}/host` volume mount in `POLARIS_GEM_Simulator/setup/docker-compose.yaml`.

5 How to Run

Three independent ways, fastest to most realistic.

5.1 Standalone MPPI test (no ROS, no sim)

```
cd ~/UIUC/AVSE/Adapt/src/adapt_mppi
python3 test/test_phase1.py
```

Prints tracking stats and writes test/phase1_result.png. Use this when changing cost terms, dynamics, or the rollout loop in isolation.

5.2 Phase 1 as a ROS 2 node (Humble, no Gazebo)

Natively on the Ubuntu 22.04 + ROS 2 Humble host:

```
cd ~/UIUC/AVSE/Adapt
source /opt/ros/humble/setup.bash
colcon build --packages-select adapt_mppi
source install/setup.bash
ros2 launch adapt_mppi adapt_phase1.launch.py
```

Needs external producers for /odom, /adapt/reference_path, /obstacles. Publishes /pacmod/steering_cmd and /pacmod/accel_cmd. Used for rosbag replays or connecting to AutoShield's stack; not useful for closed-loop dynamics without a simulator.

Runtime parameter tuning:

```
ros2 run adapt_mppi adapt_mppi_node --ros-args \
  -p v_ref:=4.0 -p K:=800 -p sigma_steer:=0.08
```

5.3 GEM Gazebo sim + MPPI bridge (Docker Noetic)

The end-to-end closed-loop path. The host (Ubuntu 22.04 + ROS 2 Humble) cannot run ROS 1 Noetic natively, so sim-side work happens inside the container.

5.3.1 Host one-time prep

```
mkdir -p ~/gem_simulation_ws/src
ln -s ~/UIUC/AVSE/POLARIS_GEM_Simulator ~/gem_simulation_ws/src/POLARIS_GEM_Simulator
cd ~/UIUC/AVSE/POLARIS_GEM_Simulator
bash setup/build_docker_image.sh
```

5.3.2 Launch the sim (container terminal A)

```
cd ~/UIUC/AVSE/POLARIS_GEM_Simulator
bash run_docker_container.sh
# --- inside the container ---
cd ~/host/gem_simulation_ws
catkin_make
```

```
source devel/setup.bash
roslaunch gem_launch gem_init.launch world_name:=track1.world vehicle_name:=e4
```

5.3.3 Launch the MPPI bridge (container terminal B)

Re-run `run_docker_container.sh` on the host to attach a second shell:

```
bash run_docker_container.sh
# --- inside the container ---
cd ~/host/gem_simulation_ws
source devel/setup.bash
roslaunch gem_mppi_sim mppi_sim.launch vehicle_name:=e4
```

Expected log line (1 Hz throttled):

```
MPPI | x=... y=... v=... -> delta=... v_cmd=... a=...
```

5.3.4 Sanity checks

```
rostopic hz /ackermann_cmd          # expect ~20 Hz
rostopic echo -n 1 /ackermann_cmd    # speed, steering_angle non-zero
rosservice call /gazebo/get_model_state "{model_name: 'gem_e4'}"
```

5.3.5 Debug fall-throughs

- **ImportError: adapt_mppi** — pass the container-side path explicitly: `roslaunch gem_mppi_sim mppi_sim.launch adapt_src:=/home/$USER/host/UIUC/AVSE/Adapt/src/ad`
- **“model not found”** from `get_model_state` — list spawned models via `rosservice call /gazebo/get_world_properties` and adjust `vehicle_name`.
- **Vehicle doesn’t move but /ackermann_cmd is publishing** — initial $v \approx 0$ clips the integrated speed to 0; raise `v_ref` (e.g. `v_ref:=4.0`).

6 What to Do Next

Prioritized, near-term to long-term.

6.1 Near term (finish the closed-loop path)

1. **Execute the host-side prep** (symlink + Docker image build) and run the first closed-loop lap on track1.world.
2. **Tuning pass.** Sweep K , H , σ_δ , σ_a , and the w_* cost weights with the first run in hand. Watch effective sample count (MPPI.effective_sample_count) — keep $ESS/K \in [0.05, 0.5]$ in steady state.
3. **Diagnostic topic.** Publish per-tick cost, ESS, and selected (δ, a) from the bridge for offline analysis and an RViz overlay.

6.2 Medium term

4. **Live obstacles.** Subscribe to /front_laser_points (sensor_msgs/LaserScan), cluster into centroids, feed self.obstacles. MPPI._obstacle_cost is already wired — only the producer is missing.
5. **Waypoints for other worlds** (parking.world, highbay_track.world) via POLARIS_GEM_Simulator/utils/generate_waypoints.py; expose via the bridge's waypoints_csv launch arg.
6. **Dynamics-parity check.** Log Gazebo ground truth vs. kinematic rollout to quantify divergence under realistic steering and speed lag.

6.3 Next design phase (Phase 2 proper)

7. **Pedestrian-prediction integration.** Swap single-snapshot obstacles for a time-indexed prediction tensor of shape $(M, H, 2)$. Obstacle cost becomes $C_{k,h}^{obs} = \sum_m (\max(r_c - \|p_{k,h} - o_{m,h}\|, 0))^2$ — same NumPy broadcast, different shape. MPPI will then delay, swerve, or creep based on *predicted* pedestrian occupancy at each rollout step, rather than reacting to a snapshot. This is the capability Stanley cannot express.

6.4 Long term

8. **Uncertainty-weighted obstacle costs** using prediction covariance — per-obstacle, per-step r_c scaling.
9. **Safety state-machine integration** with AutoShield's high-level decision module (emergency-brake override).
10. **GPU rollouts** (CuPy / torch backend) if K scales beyond ~ 4000 and CPU vectorization becomes the bottleneck.

6.5 Aspirational research track

- **Diffusion policy for pedestrian motion prediction.** Replace AutoShield’s current predictor with a conditional diffusion model that samples multi-modal futures, then feed the samples directly into MPPI’s $(M, H, 2)$ obstacle tensor (one prediction sample per obstacle, or a per-mode weighted cost). Stretch goal — may not land in the current project window. Timeline risk is tracked separately from the controller deliverable; MPPI ships regardless of prediction model.

6.6 Known risks to revisit

- **Symlink + catkin.** Noetic handles symlinks in `src/` in practice, but a physical move into `~/gem_simulation_ws/src/` is the fallback if `rospack` or `catkin_make` misbehaves.
- **Acceleration → speed integration Δt .** The bridge uses `mppi.dt = 0.1 s` when integrating a into v_{cmd} , not its own loop period. Mismatch is small at `rate_hz=20`, grows if the bridge rate drops.
- **Frame assumptions.** MPPI state and waypoints both live in the Gazebo world frame. If the reference path source ever switches to GPS/geodetic coordinates, insert an explicit transform.

6.7 Tuning guide (inherited from Phase 1)

1. Validate tracking with no obstacles first. Expect sub-decimeter lateral error; otherwise raise w_{lat} or K .
2. Keep $ESS/K \in [0.05, 0.5]$. Collapse to 1-2 means λ is too small (one-sample greedy). Saturation at K means the cost landscape is flat in the sampled region (raise σ) or λ is too large.
3. Introduce obstacles. If the vehicle plows through, raise w_{obs} or r_c . If it freezes before reaching them, w_{obs} dominates w_{spd} or the obstacles are on-path with no feasible clearance.
4. Smoothness last: raise w_{ctrl} , drop σ .